Superpowers 快速入门：让 AI 编码助手真正为你工作

> 118k Stars、GitHub 热榜第一的 AI 开发框架，用流程弥补 AI 的不可靠

你是否遇到过这些困境?

- AI 生成了一堆代码，但跑不起来
- 让 AI 修 Bug，越修越多
- 每次对话都要从头解释项目背景
- AI 写的代码自己都看不懂，更不敢交给团队

问题的根源不是 AI 不够聪明，而是缺少让 AI 稳定工作的流程。

Superpowers 就是来解决这个问题的。

什么是 Superpowers?

Superpowers 是一个**AI 编码 Agent 的技能框架与方法论**。它不依赖任何 API Key 或外部服务，通过组合式的"技能"（Skills）让 AI Agent 自动遵循经过验证的开发流程。

简单说：**Superpowers 是一套让 AI 稳定输出高质量代码的"操作手册"**。

核心理念

理念	含义
测试先行	先写测试，再写代码，让 AI 看到测试失败
系统化而非临时	流程固化，不是每次靠运气
简化复杂性	最小化实现，通过小步迭代达成目标
用证据说话	验证通过才算完成，不靠"我觉得可以"

安装（3 分钟搞定）

Claude Code（推荐）

```
# 添加市场
/plugin marketplace add obra/superpowers-marketplace

# 安装核心技能包
/plugin install superpowers@superpowers-marketplace
```

其他 IDE

平台	命令
Cursor	`/add-plugin superpowers`
Codex	Fetch `.codex/INSTALL.md` 中的指令
Gemini CLI	`gemini extensions install https://github.com/obra/superpowers`

验证安装

开启新会话，输入：

```
帮我规划这个功能：用户登录系统
```

如果 AI 自动调用了相关技能（如 `brainstorming` 、 `writing-plans` ），说明安装成功。

核心工作流：7 步从想法到代码

```
想法 → 头脑风暴 → 设计评审 → 隔离工作区 → 任务规划 → 执行开发 → 代码审查 → 合并发
```

Step 1：头脑风暴（Brainstorming）

做什么：AI 通过苏格拉底式提问帮你完善想法，直到你认可设计方案。

```
你：我想给博客加个评论系统
AI：正在使用 brainstorming 技能...
AI：评论需要支持嵌套回复吗?
你：需要，最多三层
AI：回复是实时显示还是需要审核?
...
（持续提问直到设计清晰）
```

关键点：

- AI 会先了解项目背景（读取文件、分析代码）
- 一次只问一个问题
- 设计分章节展示，你需要逐节确认
- 全部确认后生成设计文档并提交到 git

Step 2：创建隔离工作区（Git Worktree）

做什么：为新功能创建独立的 git worktree，避免污染主分支。

```
AI：正在使用 using-git-worktrees 技能...
AI：创建功能分支 feature/blog-comments
```

```
AI：隔离工作区已就绪，设计文档已提交
```

好处：

- 与其他工作完全隔离
- 可以并行开发多个功能
- 出错不影响主分支

Step 3：编写实施计划（Writing Plans）

做什么：将任务分解为 2-5 分钟的微小步骤，每个步骤包含：

- 精确的文件路径
- 完整的代码示例
- 验证命令和预期输出

```
### Task 1: 创建评论数据模型

**文件：**
- 创建：`models/comment.py`
- 测试：`tests/models/test_comment.py`

- [ ] **Step 1: 编写失败的测试**

```python
def test_comment_creation():
    comment = Comment(text="测试评论", author="张三")
    assert comment.text == "测试评论"
```

- [ ] **Step 2: 运行测试，确认失败**
- [ ] **Step 3: 编写最小实现**
- [ ] **Step 4: 运行测试，确认通过**
- [ ] **Step 5: 提交代码**

Step 4：Subagent 驱动开发

做什么：AI 为每个任务启动一个独立的子 Agent，完成后进行两阶段审查。
```

主 Agent：分发 Task 1 给子 Agent 子 Agent：编写测试 → 实现代码 → 自测 主 Agent：审查子 Agent 的代码 主 Agent：分发 Task 2 给子 Agent ...

```
**优势**:
- 每个任务都是干净的上下文
- 两阶段审查确保质量
- 快速迭代，持续验证

### Step 5：测试驱动开发（TDD）

**做什么**：严格遵循红-绿-重构循环。
```

RED（红）：写一个失败的测试 GREEN（绿）：写最小代码让测试通过 REFACTOR（重构）：改善代码，保持测试通过

```
**铁律**:

> **没有先失败的测试，就不能写生产代码。**

```python
# RED：写测试
def test_user_login():
    result = login("admin", "wrong")
    assert result.error == "Invalid credentials"
```
```

```python
# 运行测试，确认失败
$ pytest test_user_login.py
FAIL: expected 'Invalid credentials', got None
```

```python
# GREEN：最小实现
def login(username, password):
    if password != "correct_password":
        return {"error": "Invalid credentials"}
```

```
    return {"success": True}
```

Step 6：代码审查（Requesting Code Review）

做什么：对照实施计划逐项检查，发现问题按严重程度分类报告。

```
## 代码审查报告

### 严重问题（必须修复）
- ❌ 未处理空用户名的边界情况

### 建议改进（可选）
- 💡 可以将错误消息提取为常量

### 通过项
- ✅ 测试覆盖率符合要求
- ✅ 遵循项目代码风格
```

Step 7：完成开发分支

做什么：最终验证，准备合并。

```
## 分支状态

✅ 所有测试通过
✅ 代码审查通过
✅ 文档已更新

## 合并选项

1. **Squash Merge**：保留干净的历史
2. **Merge Commit**：保留完整历史
3. **Rebase + Merge**：最线性的历史
```

实战示例：5 分钟搭建一个 Todo 应用

让我们用 Superpowers 的方式，完整走一遍开发流程。

1. 启动头脑风暴

```
你：帮我规划一个 Todo 应用
AI：正在使用 brainstorming 技能...

AI：在开始之前，我需要了解一下项目背景。
      你的项目是什么类型？（Web 应用 / CLI 工具 / API 服务）
```

回答几个问题后，AI 会生成完整的设计文档。

2. 生成实施计划

设计确认后，AI 自动生成类似这样的计划：

```
# Todo 应用实施计划

## Task 1: 数据模型
- 创建 `models/todo.py`
- 创建 `tests/models/test_todo.py`
- 实现基本的 CRUD

## Task 2: API 接口
- 创建 `api/todos.py`
- 创建 `tests/api/test_todos.py`
- 实现 GET/POST/DELETE 端点

## Task 3: 前端界面
- 创建 `frontend/todo-app.js`
- 实现添加、删除、完成功能
```

3. Subagent 执行

AI 自动为每个任务启动子 Agent，每个子 Agent 严格按照 TDD 流程开发。

为什么 Superpowers 能提升 AI 编程效果?

数据对比

维度	无流程 AI 编程	Superpowers
Bug 率	高（代码未经测试）	低（每步验证）
可维护性	低（临时代码）	高（计划驱动）
团队协作	难（上下文丢失）	易（设计文档持久化）
调试效率	慢（Bug 原因不明）	快（系统化调试流程）
代码复用	差（重复造轮子）	好（Skills 可组合）

技能即资产

Superpowers 的 Skills 可以积累：

- 调试过的 Bug → 形成调试经验
- 验证过的方案 → 形成最佳实践
- 审核过的代码 → 形成代码规范

你用得越多，你的 AI Agent 越懂你的项目。

下一步：深入学习

这篇入门教程覆盖了 Superpowers 的核心工作流。如果你想：

- 掌握所有 20+ 技能的高级用法
- 学习如何在团队中落地 Superpowers
- 了解如何为特定场景定制技能
- 获得真实项目的完整案例分析

推荐学习完整付费课程，包含：

1. **进阶技能详解**：systematic-debugging、dispatching-parallel-agents 等
2. **团队协作指南**：如何在多人项目中统一 AI 工作流
3. **企业落地实践**：大型代码库的分层管理策略
4. **技能开发教程**：如何编写和测试自定义技能
5. **真实案例拆解**：从需求到上线的完整流程演示

常见问题

Q：Superpowers 收费吗?

A：Superpowers 本身免费开源（MIT 许可证）。付费内容是进阶教程和定制化服务。

Q：需要学多久才能上手?

A：按本文档操作，30 分钟即可体验完整流程。熟练运用需要 1-2 周的实践。

Q：支持哪些编程语言?

A：Superpowers 是流程框架，与语言无关。支持 Python、JavaScript、TypeScript、Go、Rust 等所有主流语言。

Q：可以只用部分技能吗?

A：完全可以。Superpowers 的每个技能都是独立的，可以按需选用。

资源链接

- GitHub：github.com/obra/superpowers（118k Stars）
- Claude Code 插件：`/plugin install superpowers@superpowers-marketplace`
- 官方文档：docs.opensuperpowers.dev

「让 AI 编程从碰运气，变成稳定输出」——这是 Superpowers 的承诺。